OBRA FÁCIL · R2M

Relatório para o backend

O que já está pronto no app e o que precisa ser feito no servidor e no banco de dados para publicar na Google Play.

Data: 19/10/2026 **App:** versão 1.0.0 (Expo SDK 57) **Pacote:** com.r2m.obrafacil

API: obra-facil-backend-n4du.onrender.com

1. Resumo

O frontend está pronto para a loja. As telas abaixo já chamam o servidor e só passam a funcionar de verdade depois que o backend estiver de acordo com este documento. Os itens estão em ordem de prioridade:

BLOQUEIA A LOJA o Google pode reprovar o app ou a função não funciona IMPORTANTE segurança e consistência DEPOIS quando os planos pagos forem lançados

#	PENDÊNCIA	ROTA	PRIORIDADE
1	Exclusão de conta com confirmação de senha, apagando todos os dados	`DELETE /user/:id`	BLOQUEIA
2	Recuperação de senha por código no e-mail	`POST /auth/forgot-password` `POST /auth/reset-password`	BLOQUEIA
3	Servidor sempre ligado (sem "dormir") e HTTPS	`Render`	BLOQUEIA
4	Conta de teste para o revisor do Google	`Banco de dados`	BLOQUEIA
5	Cada usuário só acessa os próprios dados (checar dono em todas as rotas com `:id`)	`todas`	IMPORTANTE
6	Troca de senha confirmando a senha atual	`PUT /user/:id`	IMPORTANTE
7	Renovação da sessão (token)	`POST /auth/refresh`	IMPORTANTE
8	Plano do usuário e bloqueio da troca de plano sem pagamento	`GET /subscription/me` `PUT /subscription/:id`	IMPORTANTE
9	Limites dos planos aplicados também no servidor	`POST /client, /orcamento, /obra`	IMPORTANTE
10	Registro do aceite dos termos no cadastro	`POST /auth/register`	IMPORTANTE
11	Proteções: limite de tentativas, CORS, senhas, logs	`geral`	IMPORTANTE
12	Validação das compras do Google Play	`nova rota`	DEPOIS

Padrão de erro esperado pelo app: em qualquer falha, responder com o código HTTP correto e um JSON `{ "message": "texto para o usuário" }`. O app mostra esse `message` direto na tela, então ele deve estar em português e ser compreensível.

2. Pendências em detalhe

1 Exclusão de conta

BLOQUEIA A LOJA

O Google exige que o usuário consiga excluir a conta **dentro do app** e que os dados sejam realmente apagados. No app (Configurações → Excluir conta), o usuário digita "EXCLUIR" e a senha, e o app envia:

```
DELETE /user/:id
Authorization: Bearer <token>
Content-Type: application/json

{ "senha": "senha-do-usuario" }
```

O BACKEND DEVE

- Conferir que `:id` é o usuário do token
- Validar a senha (comparar com o hash)
- Apagar o usuário e **todos** os dados dele: clientes, orçamentos, obras, assinatura, códigos e tokens
- Invalidar a sessão

RESPOSTAS

- 200 excluído → o app sai e mostra "Sua conta foi excluída"
- **401** `{"message":"Senha incorreta."}`
- **403** se o id não for do usuário

Observação: o Express precisa do `express.json()` para ler o corpo do DELETE (ele já lê por padrão). A página pública `…/OBRA-FACIL/excluir-conta.html` também oferece a exclusão por e-mail (r2m.tec.adm@gmail.com), com prazo de 15 dias. Esses pedidos são atendidos manualmente pela R2M.

2 Recuperação de senha

BLOQUEIA A LOJA

A tela "Esqueci minha senha" já está pronta e segue o mesmo padrão da verificação de e-mail do cadastro. São 3 passos: e-mail → código de 6 dígitos + nova senha → sucesso.

```
POST /auth/forgot-password
{ "email": "usuario@email.com" }
→ 200 { "message": "Se o e-mail estiver cadastrado, enviaremos um código." }

POST /auth/reset-password
{ "email": "usuario@email.com", "codigo": "123456", "novaSenha": "nova-senha" }
→ 200 { "message": "Senha alterada." }
→ 400 { "message": "Código inválido ou expirado." }
```

- Código numérico de **6 dígitos**, válido por cerca de 15 minutos e de uso único
- Nova senha com no mínimo **6 caracteres** (mesma regra do cadastro)
- Para não revelar quais e-mails existem, o `forgot-password` deve responder 200 mesmo para e-mail não cadastrado
- Limitar tentativas (ex.: 5 códigos errados bloqueiam aquele código)
- Depois de trocar a senha, invalidar as sessões antigas (recomendado)

Se os caminhos ou nomes dos campos forem outros, basta avisar: no app eles mudam em um único lugar (`frontend/src/services/api.ts`).

3 Servidor sempre disponível

BLOQUEIA A LOJA

No plano gratuito, o Render desliga o servidor quando ninguém usa, e ele leva cerca de 1 minuto para voltar. O revisor do Google vê o app "travado" na abertura e pode reprovar. Hoje o app espera até 60 s e depois mostra um aviso de conexão.

- Passar o serviço para um **plano pago** do Render (instância sempre ligada) antes de enviar para revisão
- Manter HTTPS (o Android bloqueia HTTP sem criptografia)
- Fazer backup do banco de dados

4 Conta de teste para o revisor

BLOQUEIA A LOJA

O Play Console pede um e-mail e uma senha para o revisor entrar no app. Criem uma conta **já com o e-mail verificado**, com alguns clientes, orçamentos e obras de exemplo. Ela não pode ser apagada nem ter a senha trocada durante a revisão.

5 Cada usuário acessa só os próprios dados

IMPORTANTE

Todas as rotas com `:id` precisam conferir se o registro pertence ao usuário do token. Isso vale para `/user/:id`, `/client/:id`, `/orcamento/:id`, `/orcamento/client/:id`, `/obra/:id` e `/obra/client/:id`, em GET, PUT e DELETE. Sem essa conferência, um usuário pode ler ou apagar dados de outro trocando o id na requisição. Se o registro não for do usuário, responder **404** (ou 403).

Também **não aceitar do app** campos que só o servidor pode definir: `_id`, dono (`userId`), plano e datas de criação.

6 Troca de senha com a senha atual

IMPORTANTE

Em Segurança, o app pede a senha atual e a nova. A partir desta versão, envia também a **senha atual**:

```
PUT /user/:id
{ "nome": "...", "email": "...", "CPF": "...", "CNPJ": "...",
  "senha": "nova-senha", "senhaAtual": "senha-atual" }
```

- Quando vier `senha`, exigir `senhaAtual` e compará-la com o hash. Se estiver errada: **401** `{"message":"Senha atual incorreta."}`
- Gravar a nova senha sempre como hash (bcrypt/argon2) e nunca devolver a senha nas respostas
- O campo `senhaAtual` é novo. Se a validação do servidor rejeitar campos desconhecidos, ela precisa permitir esse campo

7 Renovação da sessão

IMPORTANTE

Quando qualquer rota (fora de `/auth/*`) responde **401**, o app tenta renovar a sessão **uma vez** e repete a requisição. Se a renovação falhar, leva o usuário ao login com o aviso "Sua sessão expirou".

```
POST /auth/refresh
Authorization: Bearer <token atual, possivelmente expirado>
→ 200 { "accessToken": "novo-token" }
→ 401 se não for possível renovar
```

- Token inválido ou expirado deve retornar **401**. O 403 fica para "sem permissão"
- Duas opções de renovação: aceitar o token expirado com assinatura válida dentro de um prazo (ex.: 30 dias), ou usar um refresh token em cookie seguro (o app já envia cookies: `withCredentials: true`)
- O `/auth/login` deve continuar devolvendo `accessToken`

8 Plano do usuário

IMPORTANTE

Os planos e seus limites ficam no app (`frontend/src/config/plans.ts`). O app só pergunta ao servidor **qual é o plano do usuário**. Se não houver resposta, usa o Gratuito.

```
GET /subscription/me
→ 200 { "plano": { "id": "gratuito" }, "status": "ativa", "renovaEm": null }
```

- Valores de `id`: `gratuito`, `basico`, `profissional`. Também são aceitos os formatos `{"planoId":"basico"}` e `{"plano":"basico"}`
- Todo usuário novo começa no **gratuito**
- **Segurança:** enquanto não houver pagamento, a rota `PUT /subscription/:planoId` não pode deixar o usuário trocar para um plano pago. Ela deve ser desativada ou restrita a administradores; senão, qualquer pessoa ganha o plano Profissional de graça chamando essa rota. Hoje o app não a usa.

9 Limites dos planos no servidor

IMPORTANTE

O app já bloqueia a criação quando o usuário chega ao limite e mostra o convite para mudar de plano. O servidor precisa aplicar **os mesmos limites**, porque o bloqueio só no app pode ser contornado.

PLANO	PREÇO/MÊS	CLIENTES	ORÇAMENTOS	OBRAS	PRODUTO GOOGLE PLAY
Gratuito	R$ 0	10	10	3	—
Básico	R$ 19,90	50	100	20	`obrafacil_basico_mensal`
Profissional	R$ 39,90	sem limite	sem limite	sem limite	`obrafacil_profissional_mensal`

- **Como contar (igual ao app):** clientes = todos; orçamentos e obras = apenas os **não arquivados** (`arquivado != true`)
- Conferir em `POST /client`, `POST /orcamento` e `POST /obra`. Também conferir ao **desarquivar** (`PUT` com `{"arquivado": false}`)
- Ao bater no limite: **403** `{"message":"Você atingiu o limite de 3 obras do plano Gratuito.","code":"PLAN_LIMIT"}`

10 Aceite dos termos

IMPORTANTE

A LGPD pede prova do consentimento. No cadastro, o app **obriga** o usuário a marcar "Li e aceito os Termos de Uso e a Política de Privacidade" antes de criar a conta. O `POST /auth/register` hoje envia `nome, email, senha, CPF/CNPJ`.

- Criar no usuário os campos `aceiteTermosEm` (data/hora) e `versaoTermos`
- Até o app enviar esses campos, gravar a data do cadastro como data do aceite
- Quando estiverem prontos, o app passa a enviar `{ "aceiteTermos": true, "versaoTermos": "2026-10-19" }`. Avise para eu ligar isso

11 Proteções gerais

IMPORTANTE

- **Limite de tentativas** (ex.: `express-rate-limit`) em login, cadastro, verificação de e-mail, reenvio de código e recuperação de senha
- **CORS:** o app nativo não depende de CORS. Se houver versão web, liberar só os domínios da R2M em vez de `*`
- Senhas sempre com hash. Tokens com validade curta e segredo forte (variável de ambiente, fora do Git)
- **Logs** sem senhas, tokens nem CPF/CNPJ completos
- Validar CPF/CNPJ também no servidor (o app já confere os dígitos verificadores)
- Responder 401 para token inválido ou ausente em todas as rotas protegidas

12 Pagamento dos planos (fase 2)

DEPOIS

Pelas regras do Google, assinaturas vendidas dentro do app Android **precisam** usar o **Google Play Billing**. O app terá a compra (react-native-iap ou RevenueCat) no ponto marcado `TODO(pagamento)` em `frontend/src/services/subscription.ts`. O servidor precisará de:

```
POST /subscription/google-play   (nome a combinar)
{ "planoId": "basico", "productId": "obrafacil_basico_mensal", "purchaseToken": "..." }
→ 200 { "plano": { "id": "basico" }, "status": "ativa", "renovaEm": "2026-11-19" }
```

- Validar o `purchaseToken` com a Google Play Developer API (conta de serviço do Play Console) antes de ativar o plano
- Receber as notificações em tempo real do Google (RTDN, via Pub/Sub) para renovação, cancelamento e reembolso, e voltar ao Gratuito quando a assinatura expirar
- Com RevenueCat, a validação fica com eles e o backend só recebe um webhook

3. Rotas que o app usa hoje

Todas as rotas, exceto as de `/auth`, recebem `Authorization: Bearer <accessToken>`. Confira se cada uma existe e responde no formato esperado.

ÁREA	MÉTODO E ROTA	USO NO APP
Auth	`POST /auth/login`	`{email, senha}` → `{accessToken}`
	`POST /auth/register`	`{nome, email, senha, CPF? | CNPJ?}`
	`GET /auth/check-email?email=`	Cadastro: verifica se o e-mail já existe
	`POST /auth/verify-email`	`{email, codigo}`
	`POST /auth/resend-verification-code`	`{email}`
	`POST /auth/refresh`	Item 7 · **novo comportamento**
	`POST /auth/logout`	Encerrar sessão
	`POST /auth/forgot-password`	Item 2 · **novo**
	`POST /auth/reset-password`	Item 2 · **novo**
Usuário	`GET /user/me`	Dados do usuário logado
	`PUT /user/:id`	Editar perfil · troca de senha (item 6, campo `senhaAtual` **novo**)
	`DELETE /user/:id`	Item 1 · corpo `{senha}` **novo**
Clientes	`GET /client/userClients · GET/PUT/DELETE /client/:id · POST /client`	CRUD de clientes (limite no POST, item 9)
Orçamentos	`GET /orcamento/userOrcamentos · GET /orcamento/client/:id · GET/PUT/DELETE /orcamento/:id · POST /orcamento`	CRUD; arquivar = `PUT {arquivado}`
Obras	`GET /obra/userObras · GET /obra/client/:id · GET/PUT/DELETE /obra/:id · POST /obra`	CRUD; arquivar = `PUT {arquivado}`
Assinatura	`GET /subscription/me`	Item 8
	`GET /plans · PUT /subscription/:id · DELETE /subscription`	Existem no código, mas **não são usadas** nesta versão (planos ficam no app)

4. O que já está pronto no app

RECURSO	SITUAÇÃO
Nome, ícones, tela de abertura e pacote `com.r2m.obrafacil`	**Pronto**
Permissões mínimas (internet e biometria) e as demais bloqueadas	**Pronto**
Login com biometria/Face ID, com o login manual se falhar	**Pronto**
Cadastro com validação de CPF/CNPJ e aceite obrigatório dos termos	**Pronto**
Telas de recuperação de senha	**Pronto**
Exclusão de conta com "EXCLUIR" e senha	**Pronto**
Sessão expirada: tenta renovar e, se falhar, volta ao login com aviso	**Pronto**
Planos Gratuito, Básico e Profissional, barras de uso e bloqueio no limite	**Pronto**
Política de privacidade e termos (LGPD) no app e na web	**Pronto**
Aviso de falta de conexão, tela de erro e logs desligados na versão final	**Pronto**
Suporte por WhatsApp (31) 98676-3652 e e-mail r2m.tec.adm@gmail.com	**Pronto**
Material da loja: textos, imagens e respostas dos formulários	**Pronto**
Compra dos planos pelo Google Play	Fase 2 (depende do item 12)

5. Como testar juntos

1. Backend publica as mudanças no Render (plano pago).
2. No app (Expo Go SDK 57 ou build de teste), percorrer: cadastro → verificar e-mail → login → criar até o limite do Gratuito → esqueci a senha → trocar senha → sair → excluir conta.
3. Conferir no banco que, depois da exclusão, não sobrou nenhum registro do usuário.
4. Testar a separação entre usuários: com o token do usuário A, tentar `GET /client/<id do usuário B>`. O esperado é 404 ou 403.

Obra Fácil · R2M · Relatório gerado a partir do código do app (branch `main`). Dúvidas sobre o frontend: Matheus.